Examen CEBD – S1 – 10 Décembre 2024

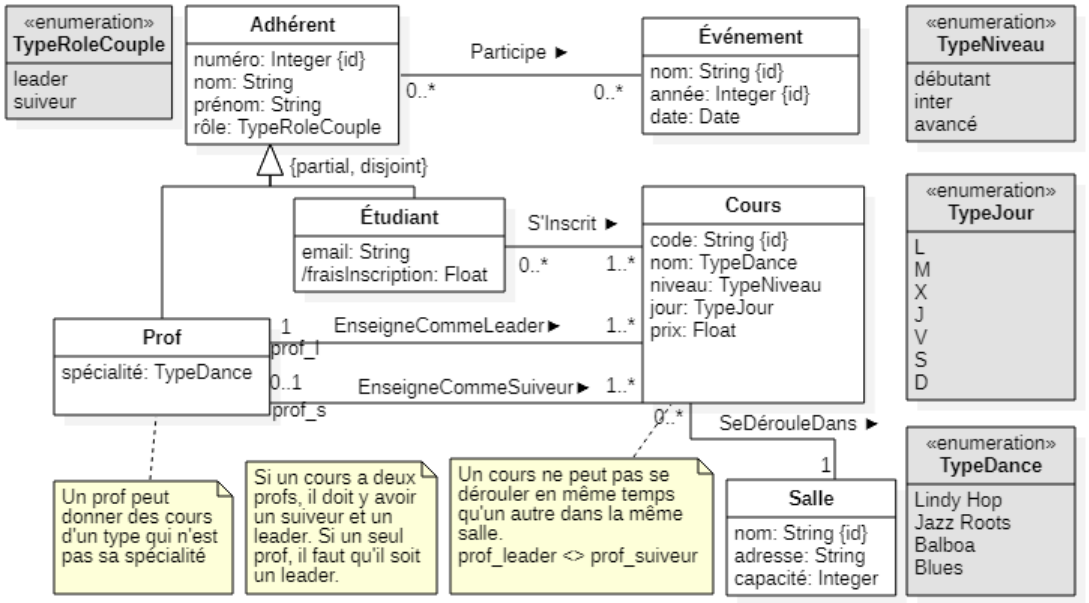
Durée : 2 Heures / Document autorisé : une feuille A4, recto-verso

Attention : Chaque partie doit être rédigée sur une **feuille à part**

A propos d'une Association de danses Swing à l'Université

Le diagramme UML ci-dessous modélise une première version d'un système d'information pour gérer les différentes activités d'une association qui vise à promouvoir les danses Swing à l'Université de Grenoble. Un début de traduction est fourni.

L'état actuel de la BD est fourni en annexe.



Adhérents (numero_adherent, nom_adherent, prenom_adherent, role_adherent)
/ (na, no, p, r) ∈ Adhérents ⟺ l'adhérent identifié par un numéro d'adhérent na a un nom no, un prénom p et un rôle r (suiveur ou leader) */*

Evénements (nom_evenement, annee_evenement, date_evenement)
/ (ne, ae, de) ∈ Evénements ⟺ l'événement nommé ne s'est déroulé lors de l'année ae avec une date de */*

Participations (numero_adherent, nom_evenement, annee_evenement)
/ (na, ne, ae) ∈ Participations ⟺ l'adhérent na a participé à l'événement ne dans l'année ae */*

Cours (code_cours, nom_cours, niveau_cours, jour_cours, prix_cours, nom_salle, prof_l, prof_s)
/ (cc, no, ni, jo, pr, ns, pl, ps) ∈ Cours ⟺ le cours identifié par un code cc a un nom no, un niveau ni, se déroule le jour jo, a un prix pr, dans la salle nommée ns, avec un professeur leader pl (au moins 1) et un professeur suiveur ps (optionnel). Un prof ne peut pas être leader et suiveur dans un même cours. */*

Etudiants_base (numero_adherent, email_etudiant)
/ (na, ee) ∈ Etudiants_base ⟺ l'étudiant identifié par un numéro na a un email ee */*

Inscriptions (numero_etudiant, code_cours)
/ (ne, cc) ∈ Inscriptions ⟺ l'étudiant ne est inscrit au cours cc */*

Profs (numero_adherent, specialite_prof)
/ (na, s) ∈ Profs ⟺ le professeur identifié par un numéro na a une spécialité s */*

Salles (nom_salle, adresse_salle, capacite_salle)
/ (ns, a, c) ∈ Salles ⟺ la salle nommée ns est située à l'adresse a et a une capacité c */*

(View) Etudiants (numero_adherent, email_etudiant, frais_inscription_etudiant)

Avec les contraintes d'intégrité suivantes (*manquent celles de l'héritage à compléter (Q1)*) :

Participations[numero_adherent] ⊆ Adherents[numero_adherent]
Participations[nom_evenement, annee_evenement] ⊆ Evenements[nom_evenement, annee_evenement]
Inscriptions[numero_etudiant] = Etudiants[numero_adherent]
Inscriptions[code_cours] ⊆ Cours[code_cours]
Cours[prof_l] ⊆ Profs[numero_adherent] ; Cours[prof_s] ⊆ Profs[numero_adherent] ; Cours[nom_salle] ⊆ Salles[nom_salle]

1 (Feuille à part) Compréhension modèle, normalisation et conception UML

Question 1 (1 point) :
Quelle stratégie a été utilisée pour la traduction de l'héritage ? Expliquer brièvement en deux phrases la signification de la contrainte {*partial, disjoint*} et donner les contraintes en relationnel (en utilisant [ ], ⊆, =, ∩, ∪)

La stratégie utilisée est *Référence*

Partial : on peut avoir des adhérents qui ne sont pas ni profs ni étudiants.
Disjoint : un étudiant ne peut pas être prof et vice-versa.

Etudiants[numero_adherent] ⊆ Adherents[numero_adherent]
Profs[numero_adherent] ⊆ Adherents[numero_adherent]
Etudiants[numero_adherent] ∩ Profs[numero_adherent] = ∅

Question 2 (2 points) :
Compléter le code SQL (SQLite) afin d'implémenter les différentes contraintes (clé, intégrité référentielle, domaine et not null). *Pas besoin de recopier la liste d'attributs fournie.*

```
-- Code Sqlite. Pas besoin de re-copier cette partie.
CREATE TABLE Cours (
   code_cours TEXT,
   nom_cours TEXT,
   niveau_cours TEXT,
   jour_cours TEXT,
   prix_cours REAL,
   nom_salle TEXT,
   prof_l INTEGER,
   prof_s INTEGER,
   ...


-- On pourrait mettre NOT NULL sur tous les attributs sauf prof_suiveur
-- si on considère toutes les informations obligatoires pour définir un cours.
CREATE TABLE Cours (
   code_cours TEXT,
   nom_cours TEXT,
   niveau_cours TEXT,
   jour_cours TEXT,
   prix_cours REAL,
   nom_salle TEXT NOT NULL, -- not null obligatoire car cardinalité 1
   prof_l INTEGER NOT NULL, -- not null obligatoire car cardinalité 1
   prof_s INTEGER,
   CONSTRAINT pk_cours PRIMARY KEY (code_cours),
   CONSTRAINT fk_cours_salles FOREIGN KEY (nom_salle)
```

```
      REFERENCES Salles(nom_salle),
   CONSTRAINT fk_cours_profs_leader FOREIGN KEY (prof_l)
      REFERENCES Profs(numero_adherent),
   CONSTRAINT fk_cours_profs_suiveur FOREIGN KEY (prof_s)
      REFERENCES Profs(numero_adherent),
   CONSTRAINT ck_cours_niveau_cours CHECK
      (nom_cours IN ('Lindy Hop', 'Jazz Roots', 'Balboa', 'Blues')),
   CONSTRAINT ck_cours_niveau_cours CHECK
      (niveau_cours IN ('débutant', 'inter', 'avancé')),
   CONSTRAINT ck_cours_jour_cours CHECK
      (jour_cours IN ('L', 'M', 'X', 'J', 'V', 'S', 'D')),
   CONSTRAINT ck_cours_prix_cours CHECK (prix_cours > 0),
   CONSTRAINT ck_cours_profs CHECK (prof_l <> prof_s)
);
```

Question 3 (4 points) :
L'association souhaite désormais inclure des informations sur l'organisation des festivals (sans billetterie) dans la base de données. Nous commençons avec un ancien fichier Excel utilisé par un adhérent pour l'organisation d'un festival précédent, qui compile des informations sur les festivals, l'adhérent responsable de chaque festival, les tarifs et les différentes salles utilisées lors des festivals. Nous adoptons ici une approche "bottom-up" en appliquant la normalisation pour intégrer les informations avec le moins de redondance possible.
On part de la relation *Festivals (nomFes, annee, type, nbPersMax, prix, numAdh, date, nomSalle)* et la liste de dépendances fonctionnelle suivantes :

nomFes, annee $\longrightarrow$ *type, numAdh, date*
type $\longrightarrow$ *nbPersMax, prix*

a. Trouver la/les clés de la relation.

   nomFes, annee et nomSalle n'apparaissent pas à droite des DFs donc ils font forcément partie de la clé. La clé de la relation Festivals est (nomFes, annee, nomSalle) car sa fermeture donne tous les attributs.

b. Évaluer la forme normale de la relation. La relation est-elle en BCNF ? Pourquoi ?

   Évaluation de la relation :

   - 1NF Oui, on a tous les attributs sont atomiques.

   - 2NF Non car certains attributs non clés dépendent partiellement de la clé (type $\longrightarrow$ nbPersMax, prix).

   La relation Festivals est donc en 1NF. Elle ne peut donc pas être en BCNF car elle n'est pas dans une forme plus restrictives (2NF).

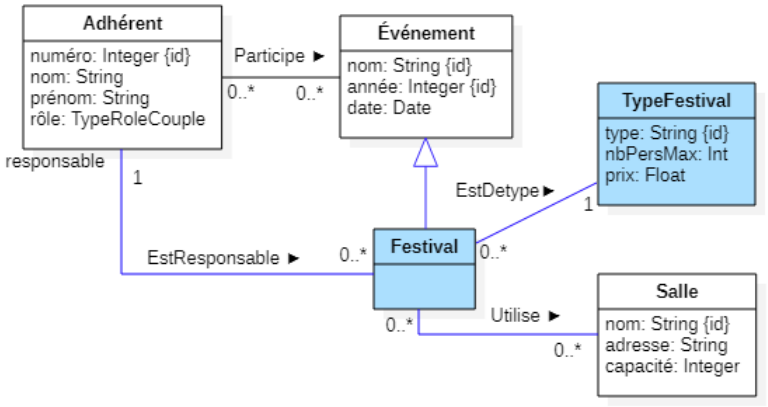
c. Proposer une décomposition en BCNF si besoin. Toute réponse sans justification sera considérée incorrecte.

   — **R1** (type, nbPersMax, prix) **BCNF**
   type $\longrightarrow$ nbPersMax, prix
   — R2 (nomFes, annee, type, numAdh, date, nomSalle)
   nomFes, annee $\longrightarrow$ type, numAdh
   (nomFes, annee, nomSalle) clé de la relation. 1FN car dépendance partielle.
      — **R21** (nomFes, annee, type, numAdh, date) **BCNF**
      nomFes, annee $\longrightarrow$ type, numAdh, date
      — **R22** (nomFes, annee, nomSalle)
      pas de DFs donc clé composée des 3 attributs. **BCNF**

   La relation Festivals est normalisé de la façon suivante :
   — **TypeFestival** (type, nbPersMax, prix)

— **Festival** (nomFes, annee, type, numAdh, date)
— **SallesFestival** (nomFes, annee, nomSalle)

Question 4 (3 points) :
En s'appuyant sur la description et DFs de l'énoncé précédent, proposer **une extension du modèle UML initial** incluant les informations sur les festivals. Proposer également une **traduction en relationnel** incluant les relations, les clés et les contraintes d'intégrité référentielles.



Avec une traduction de l'héritage par duplication cela donne une solution équivalente à la précédente :

TypeFestival (type_festival, nbPersMax_type_festival, prix_type_festival)
Festival (nom_festival, annee_festival, type_festival, numero_responsable, date_festival)
SallesFestival (nom_festival, annee_festival, nom_salle)

Festival[type_festival] ⊆ TypeFestival[type_festival]
Festival[numero_responsable] ⊆ Adherents[numero_adherent]
SallesFestival[nom_festival, annee] ⊆ Festivals[nom_festival, annee_festival]
SallesFestival[nom_salle] ⊆ Salles[nom_salle]

2 (Feuille à part) Expression de vues et requêtes en SQL

Question 5 (2 points) :
Donner le code de la vue *Etudiants* incluant les attributs propres aux étudiants plus le calcul des frais d'inscriptions en fonction du prix des cours auxquels l'étudiant est inscrit.

```
CREATE VIEW Etudiants AS
SELECT numero_adherent,
       email_etudiant,
       SUM(prix_cours) AS frais_inscription_etudiant
FROM Etudiants_base JOIN Inscriptions ON (numero_etudiant = numero_adherent)
                    JOIN Cours USING (code_cours)
GROUP BY numero_adherent, email_etudiant;
```

Question 6 (8 points) :
Exprimer en **SQL** et en **algèbre si possible** les requêtes ci-dessous. Les requêtes devront construire des résultats sans répétition de valeurs, la clause DISTINCT ne sera utilisée que lorsque nécessaire. **Pour l'algèbre, si pas possible, indiquer la raison**.

1. Donner les noms des cours du Lundi et aussi ceux du Mercredi, ainsi que le nom et la capacité de la salle dont ils se déroulent. Schéma attendu *(nom_cours, nom_salle, capacite_salle)*

   (Cours :jour_cours='L'*Salles)[nom_cours, nom_salle, capacite_salle] ∪
   (Cours :jour_cours='X'*Salles)[nom_cours, nom_salle, capacite_salle]
   ou
   (Cours :jour_cours='L' or jour_cours='X'*Salles)[nom_cours, nom_salle, capacite_salle]

   ```
   SELECT nom_cours, nom_salle, capacite_salle
   FROM Cours JOIN Salles USING (nom_salle)
   WHERE jour_cours = 'L'
   UNION
   SELECT nom_cours, nom_salle, capacite_salle
   FROM Cours JOIN Salles USING (nom_salle)
   WHERE jour_cours = 'X';

   -- OK aussi (besoin de DISTINCT potentiels doublons)
   SELECT DISTINCT nom_cours, nom_salle, capacite_salle
   FROM Cours JOIN Salles USING (nom_salle)
   WHERE jour_cours = 'L' OR jour_cours = 'X';
   ```

2. Donner les cours où il n'y a pas encore d'étudiants inscrits. Schéma attendu : *(code_cours, nom_cours)*

   Cours[code_cours, nom_cours] - (Inscriptions*Cours)[code_cours, nom_cours]

   ```
   SELECT code_cours, nom_cours
   FROM Cours
   MINUS -- EXCEPT en SQlite
   SELECT code_cours, nom_cours
   FROM Inscriptions JOIN Cours USING (code_cours);
   ```

3. Donner les étudiants adhérents 'leader' inscrits à au moins 2 cours. Schéma attendu : *(numero_adherent, nom_adherent)*

   EtudiantsLeaderInscrits ← Adherents :role_adherent = 'leader' (numero_adherent=numero_etudiant)*Inscriptions
   ELI1 ← EtudiantsLeaderInscrits
   ELI2 ← EtudiantsLeaderInscrits
   (ELI1 (EL1.numero_adherent = EL2.numero_adherent et
   EL1.code_cours <> EL2.code_cours)*ELI1)[numero_adherent, nom_adherent]

   ```
   -- Les étudiants sont les seules adhérents qui peuvent apparaître dans la table Inscriptions
   SELECT numero_adherent, nom_adherent
   FROM Adherents JOIN Inscriptions ON (numero_adherent = numero_etudiant)
   WHERE role_adherent = 'leader'
   GROUP BY numero_adherent, nom_adherent
   HAVING COUNT(code_cours) >= 2;
   ```

4. Donner pour chaque nom d'événement, le nombre total d'années où il a eu lieu ainsi que le nombre total de participants pendant toutes les années. Si un adhérent participe à un même événement deux années différentes, il sera compté deux fois. Schéma attendu : *(nom_evenement, nb_annees_evenement, nb_total_participants_toutes_annees)*. Par exemple, le *Concours de Swing* a eu lieu 3 années et il a eu 4 participants de l'association en total.

   Pas possible en algèbre car COUNT + LEFT JOIN

   ```
   SELECT nom_evenement,
   COUNT (DISTINCT annee_evenement) AS nb_annees_evenements,
   COUNT (numero_adherent) AS nb_total_participants_toutes_annees
   ```

```
FROM Evenements LEFT JOIN Participations USING (nom_evenement, annee_evenement)
GROUP BY nom_evenement;
```

5. Donner les paires de dates consécutives de la table *Evenements*. En d'autres mots, pour chaque date qui apparait dans *Evenements*, donner la date de l'événement le plus proche (dans le passé), s'il existe. Schéma attendu : (*curDate*, *prevDate*), en ordre croissant par *curDate*. Par exemple, on aurait curDate = *2023-07-11* et prevDate = *2023-05-20*.

```
SELECT e1.date_evenement AS curDate,
       MAX(e2.date_evenement) AS prevDate
FROM Evenements e1 JOIN Evenements e2 ON (e1.date_evenement>e2.date_evenement)
GROUP BY e1.date_evenement
ORDER BY e1.date_evenement;
```

```
E1 ← Evenements ; E2 ← Evenements
PlusRecents1(curDate, prevDate) ←
   (E1 (E1.date_evenement > E2.date_evenement) * E2)[E1.date_evenement, E2.date_evenement]
PlusRecents2 ← PlusRecents1
PlusPetitsPlusRecents ←
   (PlusRecents1 (PlusRecents1.curDate=PlusRecents2.curDate et
   PlusRecents1.prevDate < PlusRecents2.prevDate) * PlusRecents2)[PlusRecents1.curDate, PlusRecents1.prevDate]
PlusRecents1 - PlusPetitsPlusRecents
```

3 Annexe

code_cours	nom_cours	niveau_cours	jour_cours	prix_cours	nom_salle	prof_l	prof_s
LHI-D-L	Lindy Hop	débutant	L	150.0	Grand Swing Hall	1	2
JR-I-M	Jazz Roots	inter	M	125.0	Grand Swing Hall	3	
BA-A-X	Balboa	avancé	X	130.0	Jive Junction	3	2
BL-D-J	Blues	débutant	J	120.0	Grand Swing Hall	3	

numero_adherent	nom_adherent	prenom_adherent	role_adherent
1	Doe	John	leader
2	Smith	Alice	suiveur
3	Brown	Emma	leader
4	Johnson	Michael	suiveur
5	Williams	Olivia	leader
6	Jones	William	suiveur
7	Garcia	Sophia	leader
8	Martinez	James	suiveur
9	Lee	Emily	leader
10	Taylor	David	suiveur

numero_adherent	specialite_prof
1	Lindy Hop
2	Balboa
3	Jazz Roots

numero_adherent	email_etudiant
5	olivia.williams@email.com
6	william.jones@email.com
7	sophia.garcia@email.com
8	james.martinez@email.com

numero_etudiant	code_cours
5	LHI-D-L
5	JR-I-M
6	LHI-D-L
7	JR-I-M
8	BL-D-J
8	LHI-D-L
8	JR-I-M

numero_adherent	nom_evenement	annee_evenement
1	Concert de Jazz	2023
1	Concert de Jazz	2024
2	Concert de Jazz	2023
2	Stage de Lindy Hop	2024
3	Stage de Lindy Hop	2024
3	Concours de Swing	2022
3	Concours de Swing	2023
3	Soirée Blues	2023
4	Soirée Blues	2023
4	Concours de Swing	2022
5	Soirée Blues	2023
10	Concours de Swing	2022

nom_evenement	annee_evenement	date_evenement
Concert de Jazz	2023	2023-05-20
Concert de Jazz	2024	2024-05-18
Stage de Lindy Hop	2024	2024-08-15
Concours de Swing	2022	2022-07-10
Concours de Swing	2023	2023-07-11
Concours de Swing	2024	2024-07-12
Soirée Blues	2023	2023-11-30
Compétition Balboa	2024	2024-04-05

nom_salle	adresse_salle	capacite_salle
Grand Swing Hall	123 Rue de la Danse	60
Swing Paradise	7 Bvd. du Jazz	30
Jive Junction	10 Rue des Swingeurs	40